

SafeDrop

Software Test Document · Iteration 1

Version 0.2 (Draft) · 21 September 2026

Team

Team Member	Role(s)	Signature	Date
Alexa Stein	Team Lead	<u>Alexa Stein</u>	<u>09/21/2026</u>
Amber Rastella	Security Lead	<u>Amber Rastella</u>	<u>09/21/2026</u>
Alex Picard	Configuration Lead	<u>Alex Picard</u>	<u>09/21/2026</u>
Mateus Silva	Requirements Lead	<u>Mateus Silva</u>	<u>09/21/2026</u>
Orelmis Toribio	QA Lead	<u>Orelmis Toribio</u>	<u>09/12/2026</u>

Revision History

Version	Author	Date	Change
0.1	Amber (draft)	9/21/2026	Initial Iteration 1 test document. Automated testing report and metrics compiled from the CI run on main at commit e4a0efd. The manual testing report is not yet filled in - see the note in that section.
0.2	Orelmis Toribio (QA Lead)	9/21/2026	Added code-structure metrics (average LOC per file, LOC per function, internal fan-out per file) to Testing Metrics. Added manual tests, and styling.

Contents

1. Testing Summary
2. Manual Testing Report
3. Automated Testing Report
4. Testing Metrics
5. AI Usage Log
6. References
7. Glossary

1. Testing Summary

Iteration 1 was tested almost entirely through automated tests, run on every push and pull request by GitHub Actions and required to pass before a branch may merge to main. The QA Lead owns the suite as a whole; each developer writes the tests for the feature they build, and the reviewer checks test coverage as part of the code review described in the SPPP. The figures throughout this document come from the CI run on main at commit e4a0efd (21 September 2026), which passed all six jobs.

921 test cases passed	100% pass rate (executed cases)	94.6% / 96.0% line coverage, backend / frontend	0 high or critical vulnerabilities
---------------------------------	---	---	--

What was tested, by category:

Category	What was tested
Unit testing	Services, middleware, policies and utilities, with their dependencies mocked. 20 of the 32 backend test files are unit tests; the largest is the checkout state machine (72 cases covering every legal and illegal transition). The frontend unit tests render components with React Testing Library against a mocked API layer.
Integration testing	The Express API exercised end to end through Supertest against a real MongoDB replica set started in memory for the run, so transactions, unique constraints and TTL indexes behave as they do in production

	rather than being mocked. 12 backend test files are integration tests.
System testing	The full stack (SPA, API and a MongoDB replica set) is brought up with Docker Compose and exercised by hand against seeded data. CI additionally builds both Docker images on every push, so a change that breaks either image fails before it merges.
Acceptance testing	Acceptance criteria for work not yet implemented are staged in the suite as bodyless test cases (see code/docs/staged-acceptance-tests.md), so a story's definition of done lives somewhere that is run and reviewed rather than in prose that can drift. They are replaced by real tests when the ticket is implemented.
Regression testing	The whole suite runs on every push and pull request, so every previously passing test is a regression test. Two security regressions found during Iteration 1 were each closed with a test that fails without the fix (SCRUM-114 and the tenant-isolation cases below).

2. Manual Testing Report

2.1 Approach

Manual testing during Iteration 1 was exploratory: features were exercised by hand against a locally seeded stack as they were built, and results were not recorded as structured test cases at the time. The QA lead reperformed the following manual tests after the fact as a final check

2.2 Test environment

The full stack runs locally with Docker Compose. From the repository root, run `docker compose up --build`; copying `.env.example` to `.env` is optional, since every variable has a development default. This starts four services:

Service	Role in manual testing
mongo	A single-node MongoDB replica set. A replica set is required because the backend wraps every state-changing action in a transaction, so each state change and its audit event commit together or not at all.
mongo-init	A one-shot sidecar that initiates the replica set once Mongo is healthy, then exits.
backend	Runs migrations, then the API at <code>http://localhost:4000</code> . <code>/health</code> returns 200 when the API and database are up and 503 when the database is

	unreachable.
frontend	The Vite dev server at <code>http://localhost:5173</code> , with <code>/api/*</code> proxied to the backend so the browser talks to a single origin.

Email. Locally nothing is emailed: `MAIL_PROVIDER` defaults to console, so password-reset emails (subject, body and reset link) are printed to the backend log. Password-reset tests read the link from `docker compose logs -f backend`.

Seeding. With the stack running, run `npm run seed` from `code/backend`. This calls `seedTwoOrgs()` (`tests/helpers/seedTwoOrgs.js`), the same fixture the integration suite uses, plus demo-only chain-of-custody history. The seed is destructive: it drops and recreates the two demo organizations on every run without prompting, and it refuses to run under `NODE_ENV=production`. Cases that change state should be followed by a re-seed before the next case.

2.3 Seed dataset

The seed creates two identically built organizations, Org A (`org-a`) and Org B (`org-b`), so every check against one has an equivalent to compare against the other. Each organization contains the following.

Users (all share the password `Correct-Horse-Battery-9`):

Role	Org A	Org B	Can do
MEMBER	<code>member@a.test</code>	<code>member@b.test</code>	Browse the catalogue, submit and cancel their own requests, see only their own requests.
APPROVER	<code>approver@a.test</code>	<code>approver@b.test</code>	Everything a member can, plus approve and deny requests and record checkout and return handoffs.
ORG_ADMIN	<code>admin@a.test</code>	<code>admin@b.test</code>	Everything an approver can, plus create, edit and retire assets, add units, manage member roles, read the audit log and read the dashboard.

Assets and units (for each organization):

Asset	Units	Unit statuses
Laptop	3	REQUESTED, OUT, HELD

Camera	3	AVAILABLE, AVAILABLE, RETIRED
Projector	2	AVAILABLE, OUT

Checkout requests (all raised by the organization's member):

Request status	Unit
PENDING	The laptop's REQUESTED unit
APPROVED	The laptop's HELD unit
CHECKED_OUT	The laptop's OUT unit (due 2026-09-24 or 2026-09-26)
CHECKED_OUT	The projector's OUT unit (due 2026-09-24 or 2026-09-26)
LOST	The camera's RETIRED unit; the request moved PENDING → APPROVED → CHECKED_OUT → LOST

Audit trail. One ORG_CREATED event from the fixture, plus nine chain-of-custody events that the seed script appends to the laptop, so its history screen shows a populated, coherent timeline.

2.4 Test case summary

ID	Test case	Type	Priority	Result
MT-01	Sign in and sign out for every role	System	High	Pass
MT-02	Create a new organization	Acceptance	High	Pass
MT-03	Forgot password, reset and change password	Acceptance	Medium	Pass
MT-04	Verify role based access to actions (admin, approver, member)	System (security)	High	Pass
MT-05	Verify Tenant isolation between organizations (assets, requests etc.)	System (security)	High	Pass
MT-06	Browse and filter the asset catalogue	Acceptance	Medium	Pass
MT-07	Create and edit an asset, add a unit	Acceptance	High	Pass
MT-08	Retire an asset (with no active units)	Acceptance	High	Pass

MT-09	Retire an asset (blocked by active units)	Acceptance	High	Pass
MT-10	Submit a checkout request; duplicate refused	Acceptance	High	Pass
MT-11	Approve and deny a request	Acceptance	High	Pass
MT-12	Record checkout and return handoffs	Acceptance	High	Pass
MT-13	Cancel a pending and an approved request	Acceptance	Medium	Pass
MT-14	View My Requests (member view)	Acceptance	Medium	Pass
MT-15	View Admin dashboard summary	Acceptance	Medium	Pass
MT-16	View Asset History	Acceptance	Medium	Pass
MT-17	Audit log filter, search and pagination	Acceptance	Medium	Pass
MT-18	Change a member's role	Acceptance	Medium	Pass

3. Automated Testing Report

Where the tests live

Backend tests are in `code/backend/tests`, split into `tests/unit` (services, middleware, policies, utilities) and `tests/integration` (routes, plus a security folder). Frontend tests are in `code/frontend/tests/unit`, covering components, hooks, services and utilities. Frameworks: Vitest throughout; Supertest for API routes without starting an HTTP server; React Testing Library for components; `mongodb-memory-server` to run a real MongoDB replica set in memory for the duration of a run. Each backend test file gets its own database inside that replica set, with the production indexes applied by the same migrations that run in deployment, and the data is wiped between tests. Playwright end-to-end tests remain a nice-to-have and were not written in Iteration 1.

How it runs

Every push and pull request runs, for both backend and frontend: lint, a formatting check, the test suite with coverage, and npm audit; plus a Docker image build for each. Coverage is gated - a run fails if line coverage falls below 80%, the target set in the SPPP quality assurance plan

(NFR-10/11). Three test files are worth calling out because they are what makes the security design checkable rather than merely stated:

Test file	Cases	What it proves
<code>tests/integration/security/routePermissions.test.js</code>	134	Walks the built route table and checks every registered route against the permission matrix, including that each returns 401 without a session. A newly added route cannot be left unprotected by accident (SR-1).
<code>tests/integration/security/tenantIsolation.test.js</code>	27	Seeds two organizations and confirms a token from one gets 404 - not 403 - for the other's ids, at both the repository and the route layer (SR-2). All 11 endpoint rows are active; six that were still flagged unimplemented after their endpoints had shipped were enabled during Iteration 1, and all six passed.
<code>tests/unit/services/checkout.transitions.test.js</code>	72	Every legal and illegal transition of the checkout state machine, including what each does to the physical unit.

4. Testing Metrics

4.1 Test results and coverage

The test and coverage figures in the table below are from the CI run on main at commit e4a0efd (21 September 2026), which passed. The SPPP quality assurance plan sets line coverage at 80% or better; both packages are comfortably above it, and CI fails the build if either drops below.

Metric	Backend	Frontend
Test files	31 passed, 1 skipped (32)	27 passed, 1 skipped (28)
Test cases	679 passed, 6 skipped (685)	242 passed, 1 todo (243)
Test case pass rate	100% of executed cases	100% of executed cases
Line coverage (target >= 80%)	94.56%	95.97%
Statement coverage	94.46%	95.79%

Branch coverage	86.72%	89.97%
Function coverage	95.59%	95.26%
Dependency audit (SR-13)	0 vulnerabilities at high or above	0 vulnerabilities at high or above
Defects open against Iteration 1	SCRUM-125 (audit immutability, database layer) - open, blocked on Atlas console access	-

On the skipped and todo cases. The 6 skipped backend cases are tests/integration/security/auditDbRole.test.js, which verifies the insert-only database role for the audit collection (SR-8, SCRUM-125). They are skipped deliberately: they need a real authenticated MongoDB deployment, and run only when AUDIT_ROLE_TEST_URI points at one. They are not a gap in effort but an open item - until that role is applied, audit immutability rests on the application layer alone. The remaining skipped file and the single frontend todo are staged acceptance criteria for work not yet implemented, per the convention above.

4.2 Code structure metrics

These metrics were measured separately by the QA Lead from the source tree. LOC counts exclude comments and blank lines, and the per-file figures also exclude configuration files. Fan-out counts relative imports of the project's own modules and excludes third-party libraries.

Metric	Backend	Frontend	Combined
Average LOC / file	61.9 63 files, 3,902 LOC	63.4 55 files, 3,487 LOC	62.6 118 files, 7,389 LOC
Average LOC / function	13.6 260 functions	29.4 120 functions	18.5 380 functions
Average fan-out / file	2.6 165 internal import edges	3.1 168 internal import edges	2.8 333 internal import edges

5. AI Usage Log

This table summarizes AI contributions in this document.

Section	Who	AI contribution	Description	Verified by
---------	-----	-----------------	-------------	-------------

(%)				
1, 3, 4	Amber (on behalf of QA)	75%	Claude Code compiled the automated testing report and the metrics table from the CI run on main and from the test suite itself. Every figure was read from the CI log rather than estimated. The manual testing report was deliberately left unfilled.	Alex Picard
4.2 and layout	Orelmis	20%	The QA Lead measured the LOC, function-length and fan-out figures. Claude formatted them into a table, drafted the surrounding text, and reworked the document layout; it did not generate or estimate any figures.	Orelmis Toribio

6. References

- [1] Team 3, "SafeDrop Software Project Proposal and Plan (SPPP)," CS673, Boston University, Sept. 2026.
- [2] Team 3, "SafeDrop Software Design Document (SDD) v0.2," CS673, Boston University, Sept. 2026.

[3] SafeDrop CI run on main at commit e4a0efd, GitHub Actions, 21 Sept. 2026 - the source of every figure in Sections 3 and 4 except the code structure metrics.

[4] code/docs/staged-acceptance-tests.md - the convention for staging acceptance criteria as tests.

[5] code/docs/audit-db-role.md - the insert-only database role for the audit collection (SCRUM-125).

7. Glossary

Term	Definition
Coverage gate	A CI rule that fails the build when line coverage falls below the agreed target (80% here).
Fan-out	The number of the project's own modules that a source file imports; a measure of how tightly a file is coupled to the rest of the codebase. Third-party libraries are not counted.
Integration test	A test that exercises several layers together, here the API against a real database.
LOC (lines of code)	Non-blank, non-comment source lines.
mongodb-memory-server	A library that starts a real MongoDB replica set in memory for the duration of a test run.
Regression test	A test kept in the suite so that a behaviour which once worked cannot silently break.
Staged acceptance test	A bodyless test case standing in for an acceptance criterion not yet implemented; Vitest reports it as todo.
Supertest	A library for calling Express routes in tests without starting an HTTP server.
Unit test	A test of one module with its dependencies mocked.
Vitest	The test runner used for both packages.